

UNIT 1: System Analysis & Design, SDLC, Models, Feasibility, and COCOMO

☐ Multiple Choice Questions (MCQs)

1. **Which of the following is NOT a phase in the SDLC?**

- a) Feasibility Study
- b) Coding
- c) Data Mining ☐
- d) Maintenance

Answer: c

2. **The Spiral model is best suited for:**

- a) Small projects with clear requirements
- b) Projects with high risk and complex requirements ☐
- c) Academic projects
- d) Waterfall-based systems

Answer: b

3. **In COCOMO, cost estimation is based on:**

- a) Project deadlines
- b) Size of software in KLOC ☐
- c) Team size
- d) Programming language

Answer: b

4. **Technical Feasibility refers to:**

- a) Analyzing software licenses
- b) Checking project funding
- c) Evaluating technology & resources ☐
- d) Human resource management

Answer: c

☐ Short Answer Questions (3 Marks)

1. **Define System Development Life Cycle (SDLC).**

Answer:

SDLC is a structured process followed for developing software. It includes the following phases: Planning, Analysis, Design, Implementation, Testing, Deployment, and Maintenance. Each phase provides a foundation for the next.

2. **What is Business System Concept?**

Answer:

It refers to understanding and modeling a business environment using system thinking. It

includes identifying inputs, processes, outputs, and feedback mechanisms used in business operations.

3. **What is Technical Feasibility?**

Answer:

It is an assessment of whether the current technical resources (hardware, software, and expertise) can support the proposed system. It evaluates whether the system can be developed with the available technology.

□ **Medium Answer Questions (5 Marks)**

1. **Explain the Waterfall Model. What are its advantages and disadvantages?**

Answer:

The Waterfall Model is a linear and sequential approach to software development. It follows these stages:

- Requirements
- Design
- Implementation
- Testing
- Deployment
- Maintenance

Advantages:

- Simple and easy to understand
- Suitable for small projects with well-defined requirements
- Clear milestones

Disadvantages:

- Inflexible to changing requirements
- Testing is done late in the cycle
- Poor model for complex and long-term projects

2. **What is Cost-Benefit Analysis (CBA) in software engineering?**

Answer:

CBA involves comparing the estimated costs of developing the system against the expected benefits.

Costs include hardware, software, training, maintenance, etc.

Benefits include improved efficiency, reduced errors, increased revenue, etc.

If the benefits outweigh the costs, the project is considered viable.

□ **Long Answer Questions (8 & 15 Marks)**

□ **Q1 (8 Marks): Compare the Spiral Model with the Waterfall Model in detail.**

Answer:

Feature	Waterfall Model	Spiral Model
Structure	Linear, sequential	Iterative, risk-driven
Phases	Fixed, one after another	Repeats through planning, risk analysis, engineering, and evaluation
Flexibility	Low	High
Risk Handling	Poor	Excellent
Requirement Changes	Difficult to handle	Can accommodate changes
User Feedback	Late in development	Early and frequent

Spiral Model is best for large, complex, and high-risk projects. It emphasizes risk analysis and prototyping, which are lacking in the **Waterfall Model**.

□ **Q2 (8 Marks): Describe Feasibility Study and its types in detail.**

Answer:

A **Feasibility Study** evaluates the practicality of a proposed project. It answers the question: *"Should we proceed with the project?"*

Types:

1. **Technical Feasibility:**
Evaluates the available technology, tools, and technical expertise to build the system.
2. **Economic Feasibility:**
Also called Cost-Benefit Analysis; examines if the benefits justify the cost.
3. **Operational Feasibility:**
Checks if the system will function in the user's environment.
4. **Legal Feasibility:**
Ensures compliance with legal and regulatory requirements.
5. **Schedule Feasibility:**
Determines if the project can be completed within time constraints.

All feasibility types must be considered before development begins.

□ **Q3 (15 Marks): Explain the COCOMO model in detail with example.**

Answer:

COCOMO (Constructive Cost Model) is a software cost estimation model developed by Barry Boehm. It estimates effort (in person-months) and cost using the size of the software in **Kilo Lines of Code (KLOC)**.

Types of COCOMO:

1. **Basic COCOMO** – Simple estimation based on size
2. **Intermediate COCOMO** – Considers cost drivers
3. **Detailed COCOMO** – Includes all phases of SDLC and cost drivers

Basic COCOMO Formula:

$$\text{Effort} = a \times (\text{KLOC})^b$$

$$\text{Time} = c \times (\text{Effort})^d$$

Where a, b, c, and d vary depending on project type:

- Organic
- Semi-detached
- Embedded

Example (Organic Project):

Let size = 10 KLOC

$$\text{Effort} = 2.4 \times (10)^{1.05} \approx 25.2 \text{ person-months}$$

$$\text{Time} = 2.5 \times (25.2)^{0.38} \approx 9.6 \text{ months}$$

Advantages of COCOMO:

- Simple to use
- Provides a quick estimation
- Helps in budgeting and planning

Limitations:

- Assumes the project is based on KLOC
- May not be accurate for new technologies or agile models

SOFTWARE ENGINEERING – UNIT 2: System Design

Includes:

- Context Diagram & DFD
 - Problem Partitioning
 - Top-Down and Bottom-Up Design
 - Decision Tree, Decision Table, Structured English
 - Functional vs Object-Oriented Approach
-

☐ Multiple Choice Questions (MCQs)

1. **A context diagram is also known as:**

- a) Flowchart
- b) Level-0 DFD ☐
- c) ER Diagram
- d) UML Diagram

Answer: b

2. **Top-down design starts with:**

- a) Testing the components
- b) Developing code
- c) Designing subsystems
- d) Designing the overall system ☐

Answer: d

3. **Structured English is used for:**

- a) Coding
- b) Decision making
- c) Representing logic in human-readable form ☐
- d) Database schema

Answer: c

4. **Decision tables are most useful for:**

- a) Mathematical problems
- b) System implementation
- c) Logical conditions and actions ☐
- d) UI design

Answer: c

☐ Short Answer Questions (3 Marks)

1. **What is a Context Diagram?**

Answer:

A Context Diagram is a high-level Data Flow Diagram (DFD) that shows the system as a

single process, along with external entities that interact with it and the data flows between them.

2. **Define Problem Partitioning.**

Answer:

Problem Partitioning involves breaking a complex system into smaller, manageable modules. This helps simplify design, enhance maintainability, and enables parallel development.

3. **What is Structured English?**

Answer:

Structured English is a way of writing processes using simple English statements that resemble programming logic (e.g., IF, THEN, ELSE) for representing algorithms clearly without syntax errors.

□ **Medium Answer Questions (5 Marks)**

1. **Differentiate between Top-Down and Bottom-Up Design.**

Answer:

Criteria	Top-Down Design	Bottom-Up Design
Approach	Begins with overall system	Begins with small modules
Focus	System functionality	Reusability of components
Control Flow	From high-level to low-level	From components to the main system
Use Case	Useful when system requirements are known	Suitable for evolving systems

Top-down emphasizes breaking down the system, while bottom-up integrates reusable modules to form a system.

2. **What is a Decision Table? Give an example.**

Answer:

A Decision Table is a tabular representation of conditions and corresponding actions.

Condition 1 Condition 2 Action

True	True	A
True	False	B
False	True	C
False	False	D

This helps in handling complex logical conditions with clarity.

□ Long Answer Questions (8 & 15 Marks)

□ Q1 (8 Marks): Explain Data Flow Diagram (DFD) with Levels and Symbols.

Answer:

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through a system.

Symbols Used in DFD:

- **Process:** Circle or rounded rectangle – indicates transformation of data
- **Data Flow:** Arrow – shows direction of data movement
- **Data Store:** Open-ended rectangle – stores data
- **External Entity:** Square – represents actors (users or other systems)

Levels of DFD:

1. **Context Diagram (Level-0)**
 - Shows system as one process with external entities
2. **Level-1 DFD**
 - Breaks down the single process into sub-processes
3. **Level-2 and Beyond**
 - Further decomposes sub-processes if needed

Example:

A Library System's Level-0 DFD might show entities like Student, Librarian, and Book System interacting with the central system. Level-1 breaks it into processes like Borrow Book, Return Book, Search Catalog.

Benefits:

- Visual representation
 - Improves understanding
 - Helps in requirement analysis
-

□ Q2 (15 Marks): Compare Functional and Object-Oriented Design Approaches in detail.

Answer:

1. Functional Design

Focuses on:

- Functions and procedures
- Data flows between functions
- Separation of data and behavior

Tools:

- Data Flow Diagrams (DFD)
- Structured Charts
- Pseudo-code or Structured English

Example:

In a payroll system, individual functions like `calculateSalary()`, `deductTax()`, `printPayslip()` are defined.

2. Object-Oriented Design

Focuses on:

- Objects that encapsulate data and behavior
- Real-world modeling using classes and inheritance
- Interaction among objects through message passing

Tools:

- UML Class Diagrams
- Sequence Diagrams
- Use Case Diagrams

Example:

In a payroll system, classes like `Employee`, `Payroll`, `Payslip` are defined, with attributes and methods like `generatePayslip()`.

Comparison Table:

Aspect	Functional Design	Object-Oriented Design
Focus	Functions	Objects (data + behavior)
Reusability	Low	High
Modularity	Procedure-based	Class-based
Maintenance	Difficult	Easier
Tools	DFD, Structured Charts	UML, Class Diagrams

Aspect	Functional Design	Object-Oriented Design
Real-World Mapping	Weak	Strong

When to Use Which?

- **Functional:** Best for simple, calculation-intensive applications
- **OOP:** Best for large, complex systems with real-world modeling needs

SOFTWARE ENGINEERING – UNIT 3: Coding, Documentation & Testing

UNIT Topics:

- Structured Programming, OO Programming
 - Information Hiding, Code Reuse
 - System Documentation
 - Testing Levels, Integration Testing, Test Case Specification
 - Reliability, Verification & Validation
 - Software Metrics, Monitoring & Control
-

☐ Multiple Choice Questions (MCQs)

1. **Structured programming encourages:**
 - a) Use of GOTO statements
 - b) Hierarchical flow of control ☐
 - c) No function calls
 - d) None of the above**Answer: b**
 2. **In object-oriented programming, information hiding is achieved using:**
 - a) Polymorphism
 - b) Inheritance
 - c) Encapsulation ☐
 - d) Abstraction**Answer: c**
 3. **Validation ensures that the software:**
 - a) Meets the customer's requirements ☐
 - b) Is error-free
 - c) Is well-tested
 - d) Has no security issues**Answer: a**
 4. **Which of the following is a white-box testing technique?**
 - a) Boundary value analysis
 - b) Equivalence partitioning
 - c) Control flow testing ☐
 - d) Black box testing**Answer: c**
-

☐ Short Answer Questions (3 Marks)

1. **What is Structured Programming?**

Answer:

Structured Programming is a programming paradigm aimed at improving clarity, quality, and development time. It uses control structures such as sequence, selection (if/else), and iteration (loops) instead of arbitrary jumps like GOTO.

2. **Define Information Hiding.**

Answer:

Information hiding is a principle where internal object details are hidden from the outside. Only essential features are visible. It's achieved via encapsulation and access modifiers in OOP.

3. **What is Integration Testing?**

Answer:

Integration Testing is a level of software testing where individual modules are combined and tested as a group to expose interface defects between the modules.

□ **Medium Answer Questions (5 Marks)**

1. **Differentiate between Verification and Validation.**

Answer:

Aspect	Verification	Validation
Purpose	Ensures product is built correctly	Ensures correct product is built
Technique	Reviews, walkthroughs	Testing, user evaluation
Performed by	Developers	Testers and users
Timing	During development	After development

Example:

- Verification: Checking design documents
- Validation: Executing code with real data

2. **Explain Software Metrics with examples.**

Answer:

Software Metrics are standard measures to quantify software characteristics.

- **Lines of Code (LOC):** Measures code length
 - **Cyclomatic Complexity:** Measures decision paths
 - **Defect Density:** Defects per 1000 LOC
 - **Code Coverage:** Measures how much code is tested
- These metrics help in tracking progress, quality, and performance.

□ **Long Answer Questions (8 & 15 Marks)**

□ **Q1 (8 Marks): Explain Object-Oriented Programming Concepts with respect to Software Engineering.**

Answer:

Object-Oriented Programming (OOP) is a design paradigm that uses “objects” to represent data and behavior. Key concepts include:

1. **Encapsulation** – Wrapping data and methods that operate on data into a single unit (class).
2. **Abstraction** – Hiding complex details and showing only essential features.
3. **Inheritance** – Creating new classes from existing ones to reuse code.
4. **Polymorphism** – Ability to take many forms; methods behave differently based on objects.

Benefits in Software Engineering:

- Enhances modularity
- Promotes code reusability
- Improves maintainability
- Easier testing and debugging

Example:

A class `Car` with properties like `speed`, `color` and methods like `accelerate()`, `brake()`.

□ **Q2 (15 Marks): Discuss different levels of testing in detail with examples. Also explain test case specification and reliability assessment.**

Answer:

Levels of Testing:

1. **Unit Testing**
 - Tests individual functions or modules
 - Performed by developers
 - Tools: JUnit, NUnit
2. **Integration Testing**
 - Combines individual units and tests interactions
 - Approaches: Big Bang, Top-down, Bottom-up
3. **System Testing**
 - Verifies entire integrated system
 - Checks functional and non-functional requirements

4. Acceptance Testing

- Conducted by client/end-user
 - Verifies if the system meets business needs
 - Types: Alpha and Beta testing
-

Test Case Specification:

A **test case** is a set of inputs, execution conditions, and expected results.

Test Case Template:

- Test Case ID
- Description
- Preconditions
- Steps to execute
- Expected Result
- Actual Result
- Status (Pass/Fail)

Example:

- ID: TC001
 - Desc: Login with valid credentials
 - Input: Username = "admin", Password = "1234"
 - Expected Result: Redirect to dashboard
-

Reliability Assessment:

- Reliability refers to the probability that software operates without failure for a specified period.
- **Metrics used:**
 - Mean Time to Failure (MTTF)
 - Failure Rate
 - Defect Density
- **Methods:**
 - Historical data analysis
 - Statistical testing
 - Fault injection

Goal: Ensure software performs consistently under expected conditions.

SOFTWARE ENGINEERING – UNIT 4: Software Project Management

UNIT Topics:

- Project Scheduling
 - Staffing
 - Software Configuration Management (SCM)
 - Quality Assurance
 - Project Monitoring
-

☐ Multiple Choice Questions (MCQs)

1. **Which of the following is used for project scheduling?**
a) ER Diagram
b) DFD
c) Gantt Chart ☐
d) Flowchart
Answer: c
 2. **Which term refers to managing changes in software during development?**
a) Risk management
b) Change management
c) Software Configuration Management ☐
d) Version control
Answer: c
 3. **The main objective of software quality assurance is:**
a) Reduce staffing cost
b) Ensure software quality ☐
c) Improve design
d) Enhance documentation
Answer: b
 4. **Which activity is not part of project monitoring?**
a) Tracking progress
b) Quality control
c) Making changes to project scope
d) Scheduling tasks ☐
Answer: d
-

☐ Short Answer Questions (3 Marks)

1. **What is Project Scheduling?**

Answer:

Project Scheduling is the process of planning project activities with timelines, resources, and dependencies. Tools like Gantt Charts and PERT charts are used to visualize the schedule.

2. **What is Software Configuration Management (SCM)?**

Answer:

SCM involves systematically managing changes in software including version control, tracking modifications, and ensuring consistency throughout the development cycle.

3. **Mention any two objectives of Quality Assurance.**

Answer:

- Ensure that software meets required quality standards
- Detect and prevent defects early in development

□ **Medium Answer Questions (5 Marks)**

1. **Explain the purpose of Staffing in software project management.**

Answer:

Staffing refers to selecting the right people for a project based on skills and experience. It involves:

- Defining roles and responsibilities
 - Forming teams
 - Allocating human resources to different tasks
- Effective staffing ensures higher productivity, better quality, and timely project completion.

2. **What are the key elements of Software Configuration Management?**

Answer:

SCM has four major components:

- **Configuration Identification:** Selecting and labeling versions of software
 - **Configuration Control:** Approving changes to configuration items
 - **Configuration Status Accounting:** Tracking configuration changes
 - **Configuration Audit:** Ensuring the configuration complies with requirements
- Tools: Git, SVN, CVS

□ **Long Answer Questions (8 & 15 Marks)**

□ **Q1 (8 Marks): Explain Project Monitoring in software project management. How is it carried out?**

Answer:

Project Monitoring refers to the continuous tracking of a software project to ensure it is progressing as planned.

Key Activities:

1. **Tracking Progress:**
 - Use of charts, dashboards, and reports
 - Comparing actual progress with planned milestones
2. **Performance Measurement:**
 - Metrics such as schedule variance, cost variance
 - Tracking defects and resource utilization
3. **Risk Monitoring:**
 - Identify risks early
 - Take corrective actions to mitigate them
4. **Communication:**
 - Regular meetings and updates
 - Status reports to stakeholders
5. **Tools Used:**
 - Microsoft Project, Jira, Trello, Asana

Monitoring helps in early detection of delays, scope creep, or resource issues and aids in taking preventive or corrective action.

□ **Q2 (15 Marks): Discuss Software Quality Assurance (SQA) in detail. What are its components and how is it different from quality control?**

Answer:

Software Quality Assurance (SQA)

SQA is a **systematic approach to evaluate software quality** and ensure that the final product meets the specified requirements.

Objectives:

- Ensure adherence to standards and procedures
 - Prevent defects during development
 - Improve the development process
-

Components of SQA:

1. **Software Engineering Standards**

- IEEE, ISO standards define quality norms and procedures
 - 2. **Process Monitoring**
 - Ensures development follows pre-defined process models
 - E.g., Agile, Waterfall, Spiral
 - 3. **Formal Technical Reviews (FTRs)**
 - Peer reviews, walkthroughs
 - Helps detect defects early
 - 4. **Testing and Inspection**
 - Unit, Integration, System, and Acceptance testing
 - 5. **Measurement and Metrics**
 - Track performance indicators like defect density, MTBF
 - 6. **Training and Certification**
 - Upskilling the team for quality practices
 - 7. **Audits**
 - Internal or external audits to assess compliance
-

SQA vs. Quality Control

Aspect	SQA	Quality Control
Focus	Process-oriented	Product-oriented
Objective	Prevent defects	Detect and fix defects
Activities	Process definition, audits	Testing, inspection
Performed by	QA team	Testers

Tools for SQA:

- **Static Analysis Tools:** SonarQube
- **Testing Tools:** Selenium, JUnit
- **CI/CD Tools:** Jenkins, GitLab CI

Conclusion:

SQA is essential for delivering reliable, maintainable, and secure software. It complements development by integrating quality into every phase of the SDLC.

SOFTWARE ENGINEERING – UNIT 5: UML & Modeling

UNIT Topics:

- Static and dynamic models
 - Why modeling
 - UML diagrams:
 - Class diagram
 - Interaction diagram (collaboration & sequence diagram)
 - State chart diagram
 - Activity diagram
 - Implementation diagram
-

☐ Multiple Choice Questions (MCQs)

1. **Which of the following is a static model?**
 - a) Sequence Diagram
 - b) Activity Diagram
 - c) Class Diagram ☐
 - d) State Chart Diagram**Answer:** c
 2. **Which UML diagram shows object interactions over time?**
 - a) Class Diagram
 - b) Sequence Diagram ☐
 - c) State Chart Diagram
 - d) Activity Diagram**Answer:** b
 3. **What does UML stand for?**
 - a) Unified Modeling Language ☐
 - b) Unified Management Language
 - c) Unified Method Language
 - d) None of the above**Answer:** a
 4. **Which of the following diagrams is a dynamic model?**
 - a) Class Diagram
 - b) Object Diagram
 - c) State Chart Diagram ☐
 - d) Package Diagram**Answer:** c
-

☐ Short Answer Questions (3 Marks)

1. **What is the purpose of modeling in software engineering?**

Answer:

Modeling helps visualize, specify, construct, and document software systems. It improves communication among stakeholders, aids in understanding complex systems, and supports design validation before implementation.

2. **Differentiate between static and dynamic models.**

Answer:

- **Static Models:** Represent the system structure (e.g., Class Diagram)
- **Dynamic Models:** Represent system behavior and interactions over time (e.g., Sequence Diagram, State Chart Diagram)

3. **What is a class diagram in UML?**

Answer:

A class diagram represents the static structure of a system by showing classes, attributes, methods, and relationships (associations, inheritance, etc.) among classes.

□ **Medium Answer Questions (5 Marks)**

1. **Explain the significance of sequence diagrams in UML.**

Answer:

A **Sequence Diagram** models the interaction among objects over time. It represents:

- Objects (as lifelines)
- Messages exchanged between them
- The order of interaction

It is useful for detailing how operations are carried out in a use case, supporting dynamic behavior understanding.

Example:

In an online order system:

- Customer → OrderService: `placeOrder()`
- OrderService → Inventory: `checkAvailability()`
- Inventory → OrderService: `confirm()`
- OrderService → Customer: `orderConfirmed()`

2. **What are the key elements of a state chart diagram?**

Answer:

- **States:** Possible conditions of an object
- **Transitions:** Movement from one state to another
- **Events:** Triggers for transitions
- **Initial and Final States**

State Chart Diagrams are useful to model reactive systems like ATM or vending machines where an object's state changes based on events.

❑ Long Answer Questions (7/8/15 Marks)

❑ **Q1 (7 Marks): Describe the difference between Collaboration Diagram and Sequence Diagram.**

Answer:

Feature	Sequence Diagram	Collaboration Diagram
Focus	Time-sequenced message flow	Structural organization
Representation	Vertical lifelines and horizontal arrows	Objects and their links
Easier for	Understanding temporal order	Understanding object relationships
Message Order	Top to bottom	Numbered messages
Use Case	Modeling communication flow	Emphasizing structural interaction

Use Case Example:

- **Sequence Diagram:** Tracks the exact order in which `Login()`, `Validate()`, and `GrantAccess()` are executed.
 - **Collaboration Diagram:** Emphasizes which objects talk to which, and the structure of message passing, rather than order.
-

❑ **Q2 (8 Marks): Explain the purpose and components of an Activity Diagram with example.**

Answer:

Purpose:

Activity diagrams model the **workflow or business processes**. It shows how control flows from one activity to another.

Components:

- **Start (initial node)**
- **Activities** (represented by rounded rectangles)
- **Decisions and merges** (diamond)
- **Forks and joins** (parallel processing)
- **End (final node)**
- **Control Flows** (arrows)

Example: ATM Transaction

```
scss
CopyEdit
[Start] → [Insert Card] → [Enter PIN]
      ↓ (Incorrect) → [Retry]
      ↓ (Correct) → [Choose Transaction] → [Withdraw Cash]
      → [Print Receipt] → [End]
```

Usefulness:

- Describes complex workflows clearly
- Useful in business process modeling and system behavior representation

□ **Q3 (15 Marks): Explain various UML diagrams studied in Software Engineering with examples and their importance.**

Answer:

Unified Modeling Language (UML) provides a standardized way to model software systems. UML diagrams are divided into:

1. Class Diagram (Static)

- **Purpose:** Shows classes and relationships.
- **Components:**
 - Class name, attributes, methods
 - Associations, generalization, dependencies
- **Use:** Design object-oriented structure
- **Example:**

```
diff
CopyEdit
Class: Customer
+ name: String
+ email: String
+ makeOrder()
```

2. Sequence Diagram (Dynamic)

- Shows object interactions over time
- Lifelines, messages, return values
- Useful in understanding the order of operations

3. Collaboration Diagram (Dynamic)

- Focuses on object relationships and message exchange
 - Nodes represent objects; links represent relationships
 - Messages are numbered
-

4. State Chart Diagram (Dynamic)

- Models the states of an object and transitions
 - Includes initial/final states, events, actions
 - Example: Order object – Ordered → Shipped → Delivered
-

5. Activity Diagram (Dynamic)

- Describes workflows using control flows
 - Fork, join, decision, merge nodes
 - Models business processes
-

6. Implementation Diagram (Deployment/Component Diagram)

- **Component Diagram:** Shows software components and dependencies
 - **Deployment Diagram:** Physical deployment of software on hardware nodes
-

Importance of UML Diagrams:

- Standardized visual language for system design
- Bridges gap between requirement and implementation
- Enhances team communication and reduces ambiguity
- Aids in documentation and maintenance

UNIT 1: System Analysis, SDLC, Feasibility & Cost Estimation

□ System Analysis & Design

1. Define system analysis and discuss its role in software engineering.
2. What are the characteristics of a good system analyst?
3. Differentiate between logical and physical design in system design.

□ Business System Concept & SDLC

4. Explain the significance of Business System Concepts in software projects.
5. Compare each phase of SDLC with real-world examples.
6. What are the limitations of traditional SDLC models?

□ Software Process Models

7. Draw the architecture of the Spiral Model and explain each phase with an example.
8. Compare Waterfall Model and Spiral Model with respect to risk management and customer interaction.
9. Describe a real-life scenario best suited for the Spiral Model.

□ Feasibility Analysis

10. How would you conduct an operational feasibility study for an online examination system?
11. List five key questions to ask during a feasibility study.

□ Cost-Benefit Analysis & COCOMO

12. Create a cost-benefit matrix for a payroll system.
13. What are direct and indirect costs in CBA?
14. Write a short note on break-even analysis in software engineering.

□ Numerical (COCOMO)

15. **Given:** A software project of 100 KLOC using Basic COCOMO (Organic type). Calculate effort and development time.
 16. Calculate effort and duration using Intermediate COCOMO with given cost drivers.
 17. A software product has a size of 200 KLOC. Use COCOMO II to estimate cost considering reuse and platform factors.
-

□ **UNIT 2: System Design, DFD, Problem Decomposition**

□ **DFDs & Context Diagrams**

1. Draw Level 0 and Level 1 DFD for a railway reservation system.
2. Create a context diagram for an ATM transaction system.

□ **Problem Partitioning**

3. What are the advantages of problem partitioning in software design?
4. Write a short note on horizontal and vertical partitioning.

□ **Top-Down and Bottom-Up Design**

5. Compare top-down and bottom-up approaches in terms of modularity and reusability.
6. What type of system is best designed using bottom-up?

□ **Decision Tree, Decision Table, Structured English**

7. Construct a decision tree for a loan approval system.
8. Translate the following business rules into a decision table (Give complex conditions).
9. Write a structured English algorithm to compute monthly employee salary with allowances and deductions.

□ **Functional vs Object-Oriented Approach**

10. Describe a scenario where a functional approach fails but OO succeeds.
 11. How does data encapsulation differ between the two approaches?
 12. List major design differences between functional and OO software designs.
-

□ **UNIT 3: Coding, Documentation, Testing, Metrics**

□ **Structured & OO Programming**

1. What are the characteristics of good structured programming?
2. List advantages of object-oriented programming in real-time systems.

□ **Information Hiding and Reuse**

3. Explain how encapsulation supports information hiding.
4. How can inheritance enhance code reuse? Provide examples.

☐ **System Documentation**

5. What are the contents of user-level and developer-level documentation?
6. Describe the characteristics of good documentation.

☐ **Testing Techniques & Levels**

7. Differentiate between alpha and beta testing.
8. Explain path testing with an example.
9. Design a test case for an online shopping cart (with valid and invalid inputs).

☐ **Validation, Verification, Reliability**

10. List key differences between validation and verification with examples.
11. Define software reliability. What factors affect it?

☐ **Metrics, Monitoring, Control**

12. Explain the need for software metrics. What are size and complexity metrics?
13. How would you monitor a project in an Agile setting?

☐ **Numerical**

14. Given a module with 1000 LOC and 5 defects, calculate Defect Density.
 15. Calculate Cyclomatic Complexity for a given control flow graph (give a diagram).
 16. Estimate reliability using failure data for 3 modules (table format).
-

☐ **UNIT 4: Software Project Management**

☐ **Project Scheduling & Staffing**

1. Explain Gantt Chart with an example.
2. Create a PERT chart for a four-task software module.
3. What factors influence project staffing?

☐ **Software Configuration Management (SCM)**

4. What are the components of a Configuration Management Plan?
5. Differentiate between baseline and version control.
6. Explain the check-in/check-out system in SCM tools.

☐ **Software Quality Assurance (SQA)**

7. What are the key responsibilities of an SQA team?
8. Explain McCall's Quality Model with examples.

☐ **Project Monitoring & Control**

9. What is Earned Value Management (EVM)? List its components.
10. How can a project manager control cost and schedule overruns?

☐ **Numerical**

11. You are given the planned value, earned value, and actual cost of a project. Calculate Cost Performance Index (CPI) and Schedule Performance Index (SPI).
 12. Estimate total effort for a team of 5 working on a 3-month task with known productivity rate.
-

☐ **UNIT 5: UML & Modeling**

☐ **Static vs Dynamic Models**

1. Give examples of systems best modeled with static vs. dynamic diagrams.
2. Explain how a system's state behavior can be visualized.

☐ **Class Diagram**

3. Draw a class diagram for a hospital management system.
4. Identify aggregation, association, and inheritance from the diagram.

☐ **Interaction Diagrams**

5. Draw a sequence diagram for an e-commerce order placement.
6. Design a collaboration diagram for the same process.

☐ **State Chart Diagram**

7. Model a traffic light system using a state chart diagram.
8. Design a state chart for a document lifecycle (Draft → Review → Approved → Published)

☐ **Activity Diagram**

9. Construct an activity diagram for a university admission process.
10. What is the difference between fork and join nodes?

☐ **Implementation Diagrams**

11. Draw a deployment diagram for a client-server application.
12. Differentiate between component and deployment diagrams with examples.